**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

# Домашняя работа № 6

Автоматическое развёртывание с GitHub Actions

Выполнил:

Губанов Егор

Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

**Задача**

Настроить автоматическое развёртывание бэкенд-приложения на удалённый сервер при обновлении кода в выбранной ветке репозитория. Использовать GitHub Actions, проверить выполнение полного цикла и зафиксировать этапы настройки скриншотами.

**Ход работы**

**1 Исходные условия и схема обновления**

Я использовал приложение из labs/lab3, вручную развёрнутое в рамках ЛР4. Сервер 77.239.127.10 принимает запросы через nginx на порту 8081. Исходный код находится в labs/lab2, файлы серверной конфигурации — в labs/lab4.

Для автодеплоя выбрана существующая ветка lab3 репозитория egorr-gubanov/ITMO-ACS-Backend-2026-A. После push GitHub Actions проверяет конфигурацию, собирает образ, подключается к серверу по SSH и передаёт SHA коммита. Сервер развёртывает именно этот коммит и выполняет проверки API.

**2 Подготовка доступа CI к серверу**

Для автоматизации создан отдельный ключ ED25519. После согласования открытый ключ добавлен на сервер с ограничением restrict и принудительным запуском /usr/local/sbin/rent-ci-dispatch. Для него запрещены интерактивный терминал, перенаправление портов и агент SSH. Проверка подключения с произвольной командой id завершилась отказом: разрешена только команда deploy с SHA из 40 символов.

Закрытый ключ не передаётся в исходный код. Отпечаток открытого ключа сервера я сверил через доверенное подключение Termius и с результатом ssh-keyscan на локальном компьютере. Проверка StrictHostKeyChecking в CI остаётся включённой.

```
root@vm22690:~# bash -n /usr/local/sbin/rent-ci-dispatch
root@vm22690:~# stat -c '%a %U %n' /usr/local/sbin/rent-ci-dispatch /root/.ssh/authorized_keys
755 root /usr/local/sbin/rent-ci-dispatch
600 root /root/.ssh/authorized_keys
root@vm22690:~# ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
256 SHA256:Dacvzzxb+kdkRhGlRULTQY6jQL31kCR9KJrDNBW3nzA root@(none) (ED25519)
root@vm22690:~#
```

Рисунок 1. Права скрипта CI и проверка отпечатка SSH-ключа сервера

## 3 Настройка секретов GitHub Actions

В Settings → Secrets and variables → Actions я добавил DEPLOY_HOST с адресом сервера, DEPLOY_PORT со значением 22, DEPLOY_KEY с отдельным закрытым ключом CI и DEPLOY_KNOWN_HOSTS с проверенной строкой known_hosts. На странице отображаются четыре сохранённые записи, их значения скрыты.

Пароли PostgreSQL, RabbitMQ, JWT_SECRET и INTERNAL_KEY остаются в /etc/rent-lab4/app.env на сервере. Workflow не считывает и не печатает их значения.

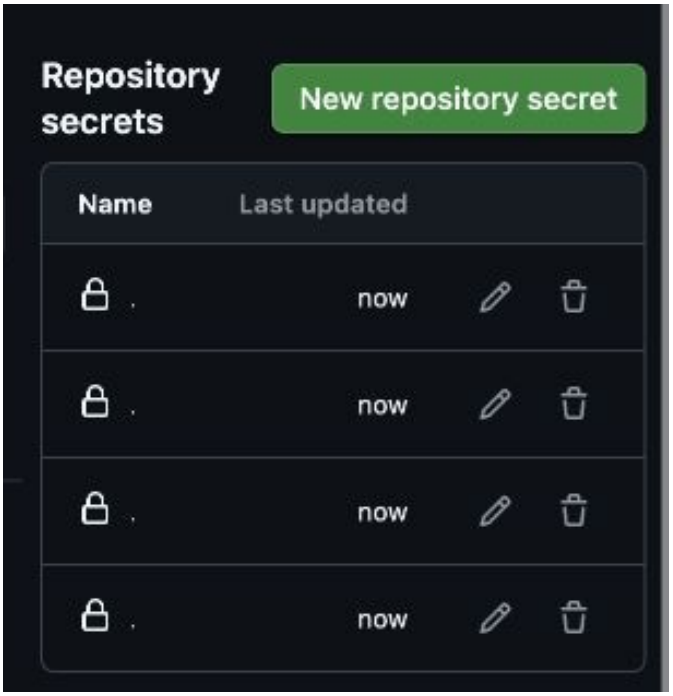



Рисунок 2. Четыре сохранённых секрета GitHub Actions

## 4 Создание workflow

В корне репозитория я создал .github/workflows/deploy-rent.yml. Триггер push ограничен веткой lab3 и путями приложения и конфигурации развёртывания. PDF, DOCX и каталог скриншотов исключены, чтобы

добавление отчётов не перезапускало сервисы. Также предусмотрен ручной триггер workflow_dispatch.

Задание validate проверяет синтаксис shell-скриптов, объединённый Docker Compose и собирает Docker-образ. Задание deploy запускается после успешного validate и выполняет удалённое развёртывание. Разрешения GITHUB_TOKEN ограничены чтением содержимого. Настройка concurrency не позволяет выполнять два развёртывания одновременно.

```
on:
  push:
    branches: [lab3]
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: rent-lab4-production
  cancel-in-progress: false
```

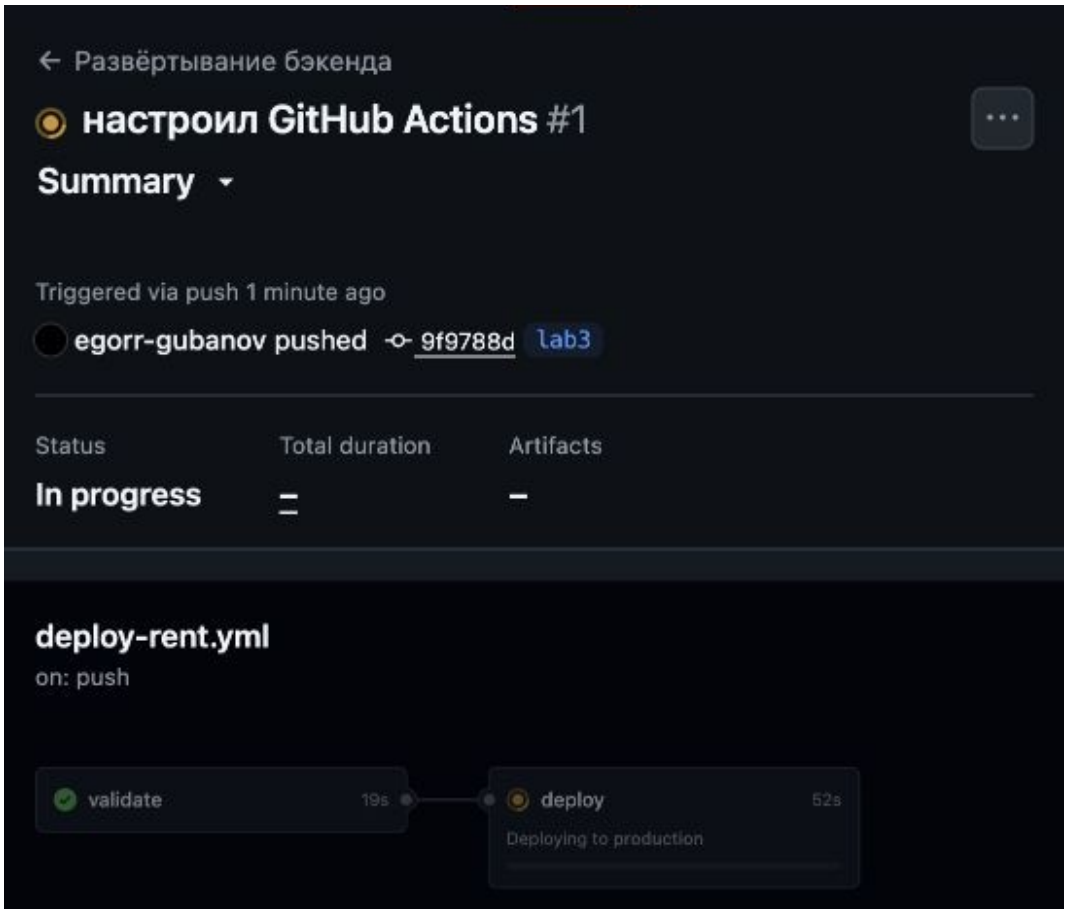



Рисунок 3. Первый запуск workflow по push в ветку lab3

## 5 Развёртывание заданного коммита

Скрипт rent-ci-dispatch проверяет формат команды и сверяет переданный SHA с текущей вершиной удалённой ветки lab3. Через git archive он извлекает исходники в /opt/rent-lab4/releases/<SHA>. Затем строится образ rent-lab4-app:<SHA>, обновляются контейнеры одного проекта rent-lab4 и ожидается состояние healthy.

После проверки API скрипт проверяет и перечитывает конфигурацию nginx, обновляет ссылку /opt/rent-lab4/current и записывает SHA, ветку и время в /var/lib/rent-lab4/deployment.json. Actions дополнительно получает публичный /deployment.json и проверяет совпадение SHA с GITHUB_SHA. Тома баз данных при обновлении сохраняются.

Первый запуск с коммитом 9f9788d завершился успешно. Оба задания получили статус Success; весь цикл занял 1 минуту 27 секунд. Во время замены контейнеров возможен короткий перерыв доступности API.

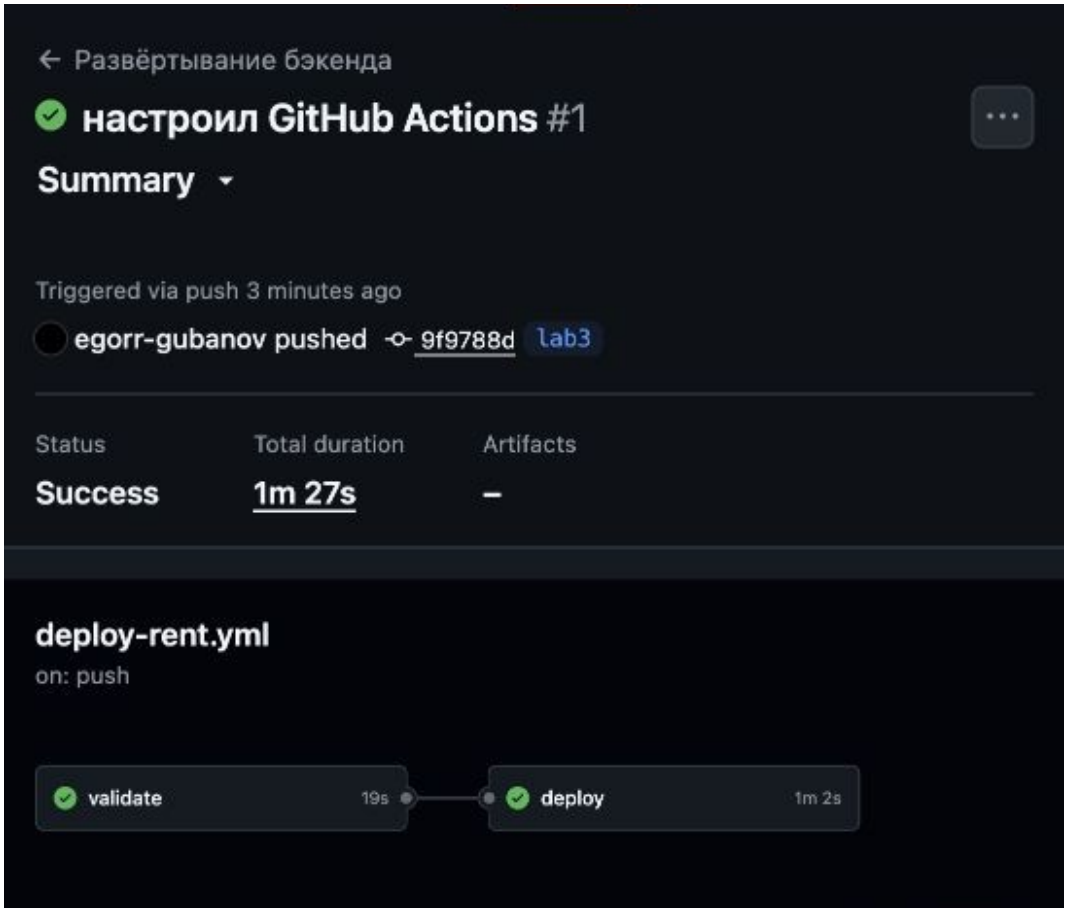


Рисунок 4. Успешное завершение первого автоматического развёртывания

## 6 Проверка триггера на изменении кода

Для проверки обновления именно исходного кода я добавил в gateway маршрут GET /healthz. До изменения этот адрес возвращал HTTP 404. Изменён только список маршрутов в labs/lab2/services/gateway/src/index.ts; новый адрес возвращает служебный ответ gateway.

Изменение сохранено от моего аккаунта коммитом bc14431 с сообщением «добавил проверку доступности API». Push автоматически запустил второй workflow. После его успешного завершения /healthz стал возвращать HTTP 200, а публичный deployment.json показал SHA bc1443118abb8a787fdbe47389e64130d1be25f6. Таким образом, проверено обновление работающего приложения после изменения кода.




Рисунок 5. HTTP 404 до обновления и предыдущая версия сервера

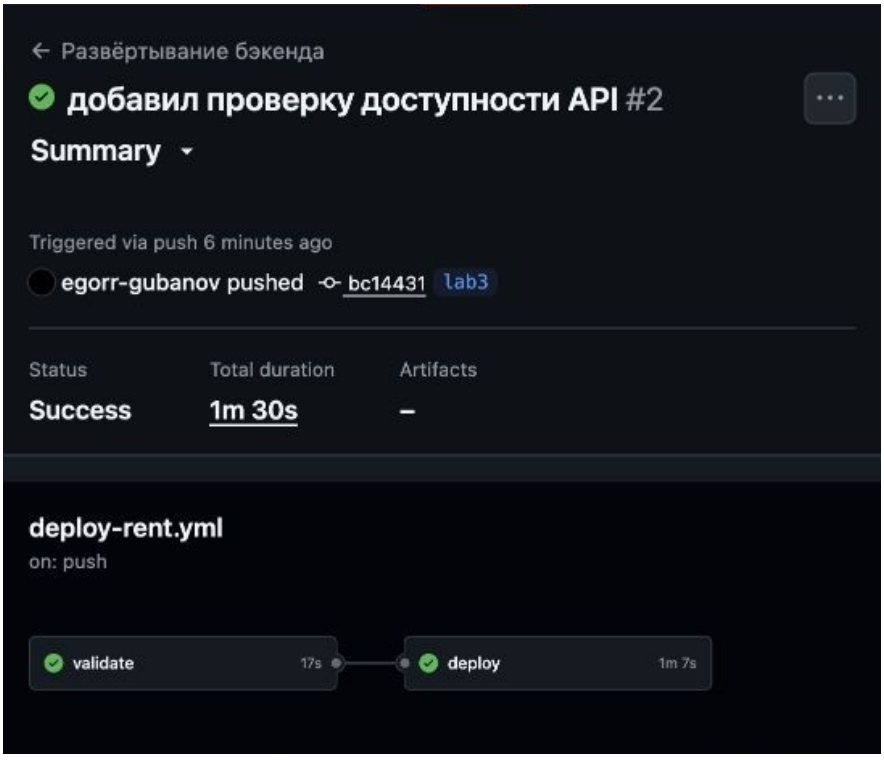



Рисунок 6. Автоматический запуск после изменения кода и успешные задания

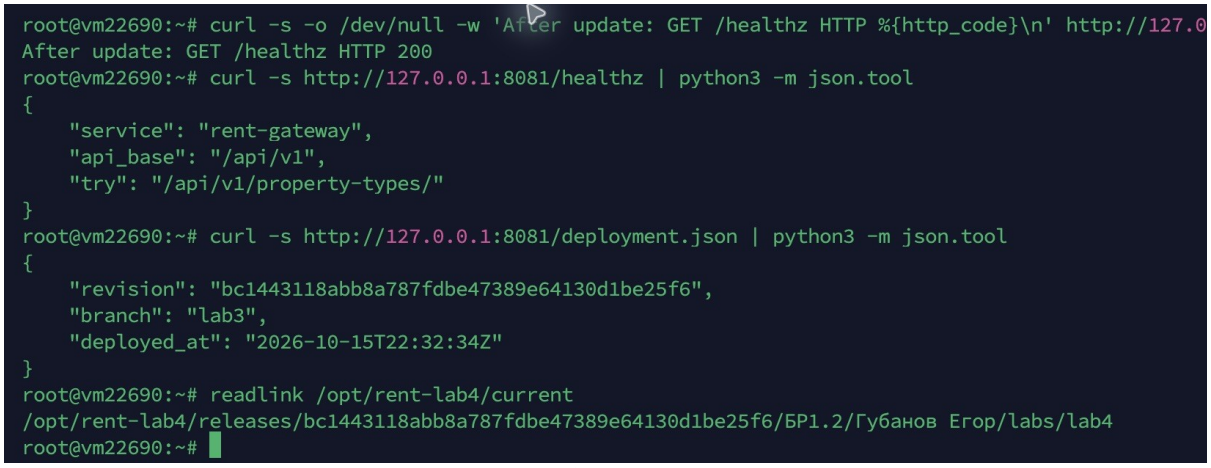



Рисунок 7. HTTP 200 после обновления и SHA нового релиза

## 7 Итоговая проверка

После повторного автодеплоя все десять контейнеров находятся в состоянии healthy. Проверки gateway, каталога из PostgreSQL и защищённого ресурса без токена проходят успешно. Скриншоты фиксируют настройку доступа и секретов, запуск workflow, результаты обоих развёртываний и изменение публичного ответа API.

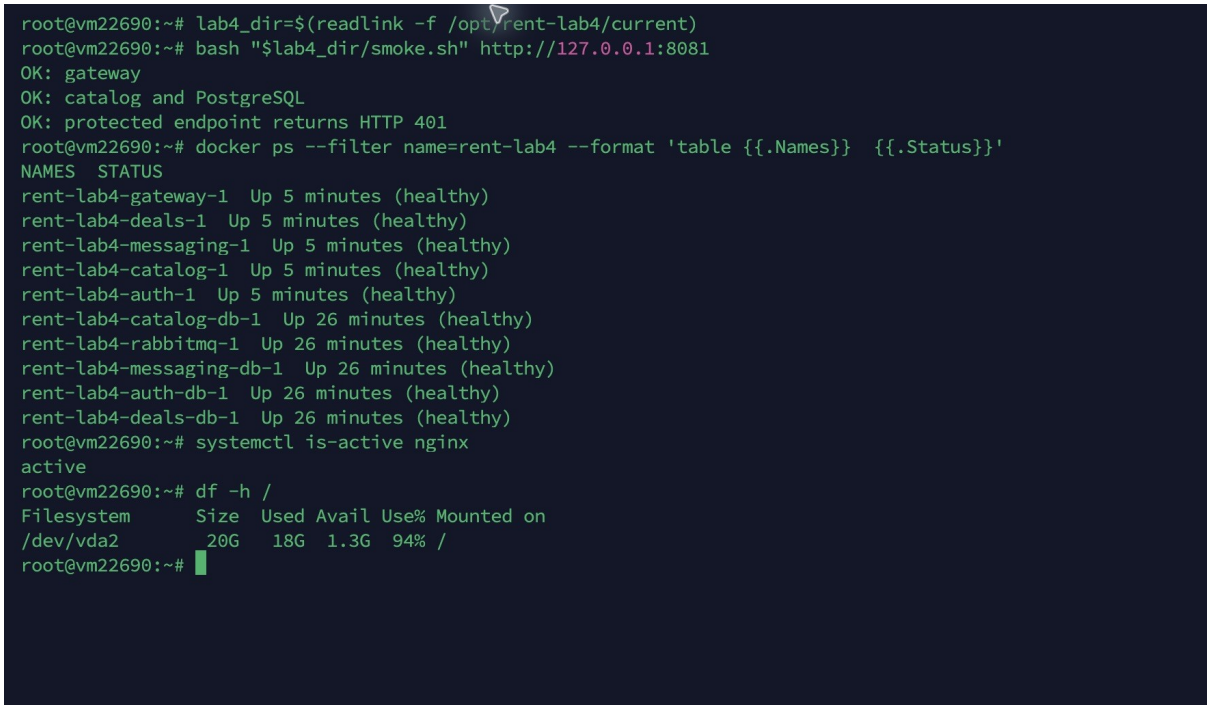



Рисунок 8. Контроль API, контейнеров и активного релиза

## Вывод

Я настроил GitHub Actions для автоматического развёртывания бэкенд-приложения при push в ветку lab3. Pipeline проверяет конфигурацию и сборку, развёртывает конкретный коммит по ограниченному SSH-ключу и проверяет публичную версию сервера. Успешно выполнены два запуска, включая обновление после изменения кода gateway. Ручное вмешательство в обновление приложения не потребовалось; данные сохраняются в томах Docker.

## Использованные материалы

GitHub Docs. События запуска workflow: https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows

Исходный код actions/checkout: https://github.com/actions/checkout

Первый запуск Actions:
https://github.com/egorr-gubanov/ITMO-ACS-Backend-2026-A/actions/runs/38817027275

Проверка обновления кода:
https://github.com/egorr-gubanov/ITMO-ACS-Backend-2026-A/actions/runs/38817652333